

Scelte progettuali

Ristrutturazione del primo progetto con Python, Django e Bootstrap

Per il secondo progetto abbiamo scelto l'opzione A. Il punto di partenza era il sito del primo progetto, realizzato in PHP e MySQL. La ristrutturazione ha riguardato soprattutto l'architettura tecnica: contenuti, funzioni, immagini e palette sono rimasti riconoscibili, mentre la logica è stata spostata in Django.

Struttura dell'applicazione

Abbiamo separato le responsabilità in modo semplice: **config** contiene le impostazioni generali; **cucina/models.py** descrive le tabelle; **views.py** gestisce ricerche, filtri e operazioni CRUD; **forms.py** valida i dati degli ingredienti; **urls.py** collega gli indirizzi alle view; i template producono le pagine HTML.

Un collegamento tipico tra URL e view è il seguente:

```
path("ricette/", views.recipe_list, name="recipe_list")
```

Dati e compatibilità con il primo progetto

I model mantengono i nomi delle tabelle e delle colonne originali tramite **db_table** e **db_column**. Per la verifica locale abbiamo scelto SQLite: il file **db.sqlite3** è incluso e non richiede un server esterno. I dump del database MySQL sono comunque conservati in **dati_sql**. Il comando **seed_data** legge gli INSERT originali e ricrea 75 ricette, 300 ingredienti, 20 regioni, 5 libri e 79 collegamenti libro/ricetta. In questo modo i dati restano verificabili e il progetto può essere ripristinato in qualsiasi momento.

Interfaccia e Bootstrap

Bootstrap 5.3.3 è servito dai file statici locali, senza CDN. Viene caricato prima del CSS del progetto; il foglio **compatibilita-bootstrap.css** risolve i conflitti con classi già esistenti. La struttura grafica segue la consegna: header a tutta larghezza, filtro e ricerca a sinistra, contenuto centrale, navigazione a destra e footer finale. Abbiamo mantenuto la palette verde e i pulsanti del sito precedente, intervenendo soprattutto su spazi, allineamenti e leggibilità.

Avvio locale e installazione offline

La cartella principale contiene **avvia.py**, **avvia.bat** e **avvia.sh**. La cartella **.venv** non viene consegnata, perché contiene file generati per il singolo computer e può essere ricostruita in modo automatico. Al primo avvio lo script controlla Python 3.12, crea **.venv**, installa i wheel presenti in dipendenze con l'opzione **--no-index**, applica le migrazioni, esegue **seed_data** e avvia Django su **127.0.0.1:8000**. La procedura funziona senza IDE e senza connessione a Internet.

```
python -m pip install --no-index --find-links dipendenze -r requirements.txt
```

Sicurezza, controlli e limiti

I form POST usano la protezione CSRF di Django. L'eliminazione degli ingredienti accetta solo POST; la modifica del nome su più ricette è racchiusa in una transazione atomica; gli ordinamenti dei libri sono limitati a campi ammessi e i collegamenti di ritorno vengono controllati prima dell'uso. I sette test automatici verificano pagine, filtri, CRUD, Bootstrap locale e ricaricamento dei dump SQL. Il progetto usa volutamente il server di sviluppo, **DEBUG** e **SQLite** perché la consegna prevede una verifica locale: non è una configurazione pensata per la pubblicazione in produzione.